Firebase, Packages, and Feature Flags

Firestore, dependency hygiene, Remote Config, and REST versus GraphQL

Dinu-Ștefan Rusu ADMD

What you should leave with

Firebase setup

flutterfire configure, a Firestore read and write behind a security rule.

Package dependencies

Read a pub.dev page, pin a version constraint, and know what pubspec.lock is for.

Feature flags

Firebase Remote Config: defaults, fetch and activate, and a kill switch.

REST and GraphQL

Over- and under-fetching, a GraphQL schema, and when each style wins.

1

Firebase and Firestore

flutterfire, Firestore, security rules, Cloud Functions

Contents Page

1 **Firebase and Firestore** 3
2 Package dependencies 12
3 Feature flags 17
4 REST and GraphQL 22

Firebase overview

Firestore

A document database with live query streams. Wired up in this lecture.

Cloud Functions

Serverless functions triggered by a write, a new user, or an HTTPS call. This lecture.

Remote Config

Feature flags and values you can change without shipping a release. This lecture.

Authentication

Sign-in providers, tokens, and the uid your rules compare against. Lecture 10.

Analytics & Crashlytics

Events and crash reports from a phone you do not own. Lecture 9.

AI (Gemini)

firebase_ai calls a hosted model from Flutter code. Lecture 12.

Firestore data model

1. A Firestore database holds collections. Each collection holds documents, and each document holds fields: strings, numbers, booleans, timestamps, maps, and arrays.
2. There is no schema the database enforces, and no joins. Two documents in the same collection can carry different fields.
3. A collection can nest inside a document, a subcollection, which is how you model one-to-many without a join: `users/{uid}/readings`.
4. **Design around the screens that read the data, not around normalized tables.**

Firestore rewards denormalization. Duplicating a field across documents to save a read is normal. Storage is cheap; an extra round trip is not.

Firebase setup in Flutter

```
# once per machine
dart pub global activate flutterfire_cli

# registers iOS/Android/web apps and writes
# lib/firebase_options.dart for you
flutterfire configure --project=my-app

flutter pub add firebase_core cloud_firestore
flutter pub add firebase_crashlytics
```

flutterfire configure runs once per project. Everything else, including the file `initializeApp` reads, is generated for you.

Initializing Firebase before runApp

```
void main() async {
  WidgetsFlutterBinding
      .ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions
        .currentPlatform,
  );
  runApp(const MyApp());
}
```

Bind first.

`ensureInitialized()` runs before any platform channel call, because plugins need the binding to exist.

Then configure.

`currentPlatform` comes from the options file `flutterfire configure` generated. Every other Firebase package assumes this ran first. Getting the order wrong is the most common first bug.

Firestore: reading and writing

```dart
final db = FirebaseFirestore.instance;

// write: merge, so other fields survive
await db.collection('readings').doc(uid).set(
  {'temp_c': 21.5, 'at': FieldValue.serverTimestamp()},
  SetOptions(merge: true),
);

// read once, and live via a Stream
final snap = await db.collection('readings').doc(uid).get();
db.collection('readings').doc(uid).snapshots();
```

Firestore security rules

```
match /readings/{userId} {
  // NOT: if request.auth != null
  // that is every signed-in user,
  // writing to everybody's doc
  allow read, write: if
    request.auth != null &&
    request.auth.uid == userId;
}
```

This runs on Google's side.
It is the only server-side check anywhere in this picture.
Never store a password or an API key in a document. Anything that needs a secret belongs behind a Cloud Function.

Cloud Functions and Firebase limits

1. Cloud Functions are the same serverless model from a FaaS platform, wired to Firebase triggers: a Firestore write, a new Auth user, a scheduled job, or an HTTPS call your app invokes directly.
2. They run in TypeScript, JavaScript, or Python, not Dart. That is a second language and a second toolchain in your repository.
3. **Use them for what the client must not do:** verifying a receipt, calling a paid API with your secret key, sending a push to someone else's device.

Where Firebase stops

Joins, aggregate reporting, arbitrary queries, and any bill you need to predict in advance. That is a job for a backend you write yourself.

Calling a Cloud Function from Flutter

```
final fn = FirebaseFunctions
    .instance
    .httpsCallable('verifyPurchase');

final res = await fn.call({
  'token': purchaseToken,
});
```

The token travels with the call.
The signed-in user's ID token goes along automatically.
The function knows who is asking without you passing a uid, and without trusting one the client sent.

The client sends a token, never an identity. The function is the only place that decides who the caller is.

2

Package dependencies

pub.dev, version constraints, pubspec.lock

Contents Page

1 Firebase and Firestore 3
2 Package dependencies 12
3 Feature flags 17
4 REST and GraphQL 22

Choosing a package

pub.dev is the registry

Every package lives here, with docs and the install command.

Read the scores critically

Likes and downloads are popularity; pub points are mechanical.

Check the last publish date

A plugin last released two years ago is maintenance you inherit.

Check the platforms

Native code often supports fewer platforms than you assume.

Open the dependency tree

You adopt every transitive dependency too, conflicts and all.

Check the license

MIT, BSD, Apache-2.0 are fine; copyleft needs a conversation first.

Adding a package

```yaml
dependencies:
  flutter:
    sdk: flutter
  http: ^1.5.0
  cloud_firestore: ^6.0.0
dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.5.0
```

```
flutter pub add http   # resolves
flutter pub get        # lockfile
flutter pub upgrade    # allowed
flutter pub outdated   # newer
```

Do not hand-edit pubspec.yaml. flutter pub add writes the constraint and resolves it in one step. Versions above are illustrative and will be stale by the time you read them.

Version constraints and caret syntax

Constraint	What it allows	What it blocks
`^1.5.0`	`>= 1.5.0` and `< 2.0.0`	2.0.0, a breaking major
`^0.3.2`	`>= 0.3.2` and `< 0.4.0`	0.4.0; before 1.0, minors may break
`1.5.0`	exactly 1.5.0, forever	every patch, including security fixes
`any`	whatever the resolver picks	nothing at all; never ship this

pubspec.yaml says what you will accept; pubspec.lock records what you actually got.

Before 1.0.0, a minor version may break you. Semantic versioning only promises stability after the first major release.

pubspec.lock and version control

1. **pubspec.yaml says what you will accept. pubspec.lock records what you actually got.**
2. Commit pubspec.lock for an application: it is what makes your build, your teammate's build, and CI resolve to identical code.
3. Do not commit it for a package you publish: there, consumers should resolve against their own constraint set.
4. flutter pub get obeys the lockfile. flutter pub upgrade rewrites it.

3

Feature flags

Firebase Remote Config, staged rollout, kill switch

Contents Page

1 Firebase and Firestore 3
2 Package dependencies 12
3 Feature flags 17
4 REST and GraphQL 22

Deployment versus release

- A staged rollout ships a binary to 1, 5, 10, 50, 100 percent, and halts if crashes spike, but a user who has it keeps it.
- A flag separates the two: the code ships to everyone, off, and a server decides who sees it.

Deploy is a build step. Release is a runtime decision.

Merge to main
The flag defaults to off.
Ship to the store
The code is on every device.
Flip the flag to 5%
You release, without a build.

The kill switch is the main benefit.

Firebase Remote Config

- A key/value store hosted by Firebase: the app ships with defaults, then fetches overrides for offline use.
- Values are typed: bool, int, double, String, or JSON, so you change content without a release.
- Conditions target by version, platform, country, or a random percentile.

Other tools: LaunchDarkly, Unleash, PostHog.

setDefaults()
baked into the binary
fetch()
download, then cache
activate()
make fetched values live
getBool()
read at the call site

Fetch and activate are two steps on purpose. Values never change underneath a screen that is already rendered.

Remote Config in Dart

```dart
final rc = FirebaseRemoteConfig.instance;
await rc.setDefaults(const {
  'checkout_v2_enabled': false,
  'feed_page_size': 20,
});
await rc.setConfigSettings(RemoteConfigSettings(
  fetchTimeout: const Duration(seconds: 10),
  minimumFetchInterval: const Duration(hours: 1),
));
await rc.fetchAndActivate();
if (rc.getBool('checkout_v2_enabled')) return const CheckoutV2();
```

Never cache the read value in a global; call getBool again at the call site, every time.

Feature flag guidelines

1. Fetches are throttled server-side, and billed per fetch above a free daily allowance since September 2026. Set minimumFetchInterval to zero only in debug builds.
2. Roll out by percentile: 1, 10, 50, then 100 percent, watching crash-free users at each step. Lecture 9 covers that metric.
3. Every live flag is a branch you must keep working. Delete the flag and the dead code path as soon as it reaches 100 percent.
4. **Flags are not security.** The disabled code still ships inside the binary, and the values are readable. Authorization still belongs on the server.

4 REST and GraphQL

REST, GraphQL, round trips

Contents Page

1 Firebase and Firestore 3
2 Package dependencies 12
3 Feature flags 17
4 REST and GraphQL 22

REST verbs and semantics

Resources named by URLs, acted on with standard HTTP methods.

Method	Safe	Idempotent	What it is for
GET	yes	yes	read a resource
POST	no	no	create; two sends make two
PUT	no	yes	replace it wholesale
PATCH	no	not in general	partial update
DELETE	no	yes	remove; twice ends the same

Safe and idempotent are different questions. PUT and DELETE are idempotent but not safe: retry freely, never a POST without an idempotency key.

Over-fetching and under-fetching

One profile screen: the user's name, their follower count, and their last 20 posts.

REST: three round trips

```
GET /v1/users/42
GET /v1/users/42/posts?limit=20
GET /v1/users/42/followers/count
```

GraphQL: one round trip

```
POST /graphql
user(id: 42) {
  name
  followerCount
  posts(limit: 20) { id title }
}
```

A round trip costs 50 to 150 ms on mobile. GraphQL moves the joins server-side, where a careless resolver turns one query into N+1 database calls.

GraphQL: schema and query

```
type User {
  id: ID!
  name: String!
  posts(limit: Int): [Post!]!
}
query ProfileScreen($id: ID!) {
  user(id: $id) {
    name
    posts(limit: 20) { id title createdAt }
  }
}
```

The schema is the contract. A query reads, and the response JSON has exactly this shape.

GraphQL mutations and schema benefits

```
mutation LikePost(
  $postId: ID!
) {
  likePost(id: $postId) {
    id
    likeCount
    likedByMe
  }
}
```

- A mutation writes, and returns the new state, for your cache.
- One endpoint, one schema, every client: iOS, Android, and web ask for the fields they render.

Everything is a POST to one URL, so HTTP caching stops helping here.

Introspection generates the tooling. GraphiQL, autocomplete, and Dart types can come straight from the schema.

GraphQL in a Flutter app

```
final link = HttpLink('https://api.example.com/graphql');
final client = GraphQLClient(
  link: AuthLink(getToken: () async => 'token').concat(link),
  cache: GraphQLCache(),
);
Query(
  options: QueryOptions(document: gql(profileQuery)),
  builder: (result, {fetchMore, refetch}) {
    if (result.isLoading) return const Spinner();
    return ProfileView(result.data!['user']);
  },
);
```

Errors arrive with HTTP 200 plus an errors array. Check `hasException`.

REST vs GraphQL comparison

Aspect	REST	GraphQL
Endpoints	one per resource	one URL, POST /graphql
Caching	HTTP caching, for free	a cache you configure
Schema	OpenAPI, if kept current	mandatory, introspectable
Errors	the HTTP status code	HTTP 200 plus an errors array

Simple resources favor REST. Related data and several clients favor GraphQL.

Summary

1. **Firestore rules are the only server-side check you get.** Validate on the client for speed, on the rules for correctness.
2. **A feature flag separates deploy from release.** The kill switch is the reason to add one.
3. **REST and GraphQL trade over-fetching for a normalized cache and a harder server.** Pick per screen, not once for the whole app.

Further reading

Firebase

firebase.google.com/docs/flutter/setup · firebase.google.com/docs/firestore

Packages

dart.dev/tools/pub/dependencies

Feature flags

firebase.google.com/docs/remote-config

APIs

graphql.org/learn · pub.dev/packages/graphql_flutter

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel